Chain-of-Agents: A Sequential Self-Correcting Framework for Generating Reliable LLM-Based Multi-Agent Systems

1 Introduction

LLM-based agents have shown promise in reasoning, planning, and tool use. Recent work evaluates LLMs as collaborators in interactive environments, highlighting both their strengths and limitations in multi-agent settings [1]. Other lines of research propose more structured communication or reasoning among multiple models, such as cross-model exchange of intermediate thoughts [2], or improved decoding and self-consistency strategies for reasoning [3, 4]. Multi-agent architectures combining different model types have also been explored for complex planning problems with explicit and implicit constraints [5].

Despite this progress, most multi-agent systems are either: (i) *manually engineered* pipelines with fixed roles and communication patterns, or (ii) *one-shot architectures*, where a single LLM is prompted to output an entire system specification (agents, roles, message flows, and control logic) in a single step.

The one-shot paradigm is fragile. It demands that the LLM correctly anticipate failure modes, communication bottlenecks, and missing capabilities without any opportunity to revisit or repair its own design. There is no explicit notion of architectural critique, refinement, or verification.

We call this the **one-shot architecture problem**: current methods treat architecture generation as a single prediction, rather than a reasoning process that can involve multiple passes of analysis, critique, and correction.

2 The One-Shot Architecture Problem

In a one-shot generator, a single LLM takes a high-level task description and outputs a complete multi-agent specification. This approach has three core limitations:

- **No structural self-correction.** Once the architecture is emitted, there is no built-in mechanism for detecting or revising inconsistent, redundant, or incomplete designs.
- **No explicit architectural reasoning.** The system does not decompose the design process into steps (e.g., task analysis, role assignment, communication design) that can each be checked or refined.
- **Brittle generalization.** The resulting architectures may work on a narrow benchmark but fail when tasks or domains change, because the design choices are not grounded in explicit decomposition or critique.

By contrast, recent work has shown that multi-step procedures, such as chain-of-thought prompting on challenging reasoning benchmarks [4], cross-model exchange of intermediate thoughts [2], and soft self-consistency for LLM agents [3], can significantly improve robustness and reliability. Our key idea is to lift these insights from the *token-level* or *solution-level* to the *architecture level*.

3 Solution: Chain-of-Agents Architecture Generator

We propose **Chain-of-Agents (CoA)**, a meta-architecture in which multiple *meta-agents* are responsible for different stages of architecture design. Instead of emitting a multi-agent system in one shot, CoA runs a short pipeline of agents that sequentially reason *about* the architecture. CoA consists of the following meta-agents:

- **Decomposer Agent.** Given a natural language task description, this agent produces a structured decomposition: subtasks, dependencies, constraints, and required capabilities (e.g., math reasoning, retrieval, verification).
- **Planner Agent.** Consumes the decomposition and proposes an initial multi-agent workflow: agent roles, communication channels, and execution order.
- **Critic Agent.** Analyzes the proposed workflow to identify missing steps, circular dependencies, redundant agents, or unclear information flow. This is analogous to critique-and-revise reasoning at the architecture level.
- **Refiner Agent.** Edits the architecture in response to Critic feedback, repairing structural flaws and improving clarity. Iterating Critic–Refiner steps yields a limited self-improvement loop.
- **Verifier Agent.** Performs a lightweight simulated execution or consistency check: does every agent have the information it needs? are all required outputs produced? are there unreachable or dead-end states?
- **Synthesizer Agent.** Converts the refined, verified architecture into a concrete representation (e.g., a configuration graph or program) suitable for execution in a multi-agent runtime.

References

[1] Guande Wu, Chen Zhao, Claudio Silva, and He He. 2024. **Your Co-Workers Matter: Evaluating Collaborative Capabilities of Language Models in Blocks World**. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 4941–4957, Bangkok, Thailand. Association for Computational Linguistics.

[2] Zhangyue Yin, Qiushi Sun, Cheng Chang, Qipeng Guo, Junqi Dai, Xuanjing Huang, and Xipeng Qiu. 2023. **Exchange-of-Thought: Enhancing Large Language Model Capabilities through Cross-Model Communication**. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15135–15153, Singapore. Association for Computational Linguistics.

[3] Han Wang, Archiki Prasad, Elias Stengel-Eskin, and Mohit Bansal. 2024. **Soft Self-Consistency Improves Language Models Agents**. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 287–301, Bangkok, Thailand. Association for Computational Linguistics.

[4] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, and Jason Wei. 2023. **Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them**. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13003–13051, Toronto, Canada. Association for Computational Linguistics.

[5] T. Karthikeyan, Om Dehlan, Mausam, and Manish Gupta. 2025. **LRPLAN: A Multi-Agent Collaboration of Large Language and Reasoning Models for Planning with Implicit & Explicit Constraints**. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 8280–8310, Suzhou, China. Association for Computational Linguistics.